Spring 26 CSCI 240 – PA 6

Priority Queues (PQ) and Heaps

Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab.

New: You can work with a lab partner and each one must submit the same PDF file (include both names in the submission file). Each person must include a brief statement about your contribution to this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Exercise 1: Use sorted list priority queue class from textbook (**SortedPriorityQueue** for C++ and **SortedListPriorityQueue** for Java) and create a test driver to perform some operations to confirm it is working correctly. Use a PQ with integer and string as entry. Create two PQ objects – one with the largest value having highest priority and one with lowest value having highest priority. **Be sure to set up and use two different comparators for the PQ entry**.

Perform the following operations: insert(5, “five”), insert(4, “four”), insert(7, “seven”), insert(1, “one”), min(), removeMin(), insert(3, “three”), insert(6, “six”), min(), removeMin(), min(), removeMin(), insert(8, “eight”), min(), removeMin(), insert(2, “two”), min(), removeMin(), min(), removeMin(). Output value (int and string) for each min() operation so there should be 6 pairs for each set of output.

Exercise 2: Use your priority queue from exercise 1 to sort data in **descending order** (use int and string as entry so store <10, “10”> for a value of 10 in the data file). Sort the data file ***small1k.txt***, containing a list of 1,000 integer values, and output the first 5 and last 5 values to the screen (just output the key with 5 values on one line with at least one space between the 2 values). Sort the data file ***large100k.txt***, containing a list of 100,000 integer values, and output the first 5 and last 5 values to the screen (just output the key with 5 values on one line with at least one space between the 2 values). For each set of data, collect actual run times in milliseconds and display them to the screen as well. Confirm that the runtime is O(n^2).

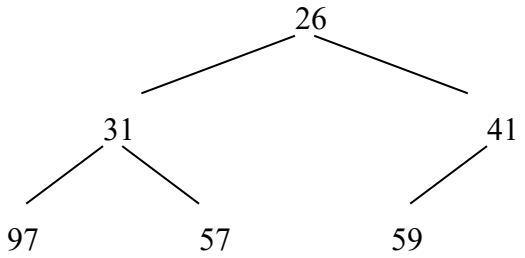
Exercise 3: Use the heap priority queue class from the textbook (**HeapPriorityQueue** for C++ and Java) and use same test driver from exercise 1 to perform some operations to confirm it is working correctly (just replace **SortedListPriorityQueue** with **HeapPriorityQueue**). Create two PQ objects – one with

the largest value having highest priority and one with lowest value having highest priority. **Reuse the two comparators from exercise 1 for the PQ entry**. You should get the same output as exercise 1.

Exercise 4: Use your priority queue from exercise 3 to sort data in **descending order** (use int and string as entry so store <10, "10"> for a value of 10 in the data file). Sort the data file ***small1k.txt***, containing a list of 1,000 integer values, and output the first 5 and last 5 values to the screen (just output the key with 5 values on one line with at least one space between the 2 values). Sort the data file ***large100k.txt***, containing a list of 100,000 integer values, and output the first 5 and last 5 values to the screen (just output the key with 5 values on one line with at least one space between the 2 values). For each set of data, collect actual run times in milliseconds and display them to the screen as well. Confirm that the runtime is O(nlogn).

Question 1: What is a PQ? Describe how you can use it to operate like a regular queue (i.e., use a PQ to do FIFO).

Question 2: Given the heap below, show the resulting heap after each of the following operations -- removeMin(), insert(30)



You can earn EC for only one of the options below:

Extra Credit Option 1: Add code to your exercise 4 to sort the data files ***large100k.txt*** in descending order using the values in the file as strings (key would be string "15" instead of integer 15 so use <string, int> as an entry like <"15", 15>). Output data to an output file as well with 5 values per line (just output the key with at least one space between the 2 values). You can submit one version here to include exercise 4. Make sure to provide the first screenshot of the output file as part of the submission.

Extra Credit Option 2: Provide pseudocode for insert() and removeMin() operations using a linked binary tree for the heap. *Hint: determine or keep track of the last node in the binary tree.*

Fill out and turn in the PA submission file for this assignment (save as PDF format).